

8. Načrtovanje IS

KAKO pripravimo načrt za izvedbo rešitev?

Faze razvoja IS



KAJ je potrebno narediti? Zahteve IS se pregledajo in strukturirajo.

KAKO bomo to naredili? Zahteve se zapišejo v formalni obliki dokumentacije (diagrami, dokumenti) in pripravi se načrt izvedbe.

Programiranje IS.

Ali obstajajo **napake** in KAJ ne dela prav?

Prenos IS v realno uporabo. Pravimo, da gre IS **v produkcijo** – za uporabo končnih uporabnikov.

Popravki in dopolnitve IS. Odpravljanje napak in nadgradnja / dodajanje novih funkcionalnosti.

Načrtovanje IS

- Glavni namen načrtovanja je **izdelati načrt zgradbe sistema glede na specifikacije**, ki so bile zbrane v fazi analize
- **Načrt** daje odgovor na vprašanje, **KAKO** izdelati sistem, da bo ustrezal zahtevam, ki smo jih evidentirali v fazi analize
- Načrtovanje je namenjeno razvijalcem, naročnik pri načrtovanju načeloma ne sodeluje
 - Razvijalci bodo razvili sistem točno glede na načrt, zato mora načrt predstaviti pravilno končno željeno stanje IS



Načrtovanje IS

- Pri načrtovanju **izhajamo iz modelov**, ki so nastali v prejšnji fazi
- **Modeli**, ki so bili narejeni zelo abstraktno, se preslikajo **v bolj podrobne načrte**
 - Lahko se uporabi enaka tehnika modeliranja
 - Največkrat se uporabi primernejša tehnika modeliranja (običajno iz nabora UML)
- Modeli, ki so bili narejeni podrobno (kar je praviloma neprimerno za prejšnjo fazo), se lahko neposredno uporabijo za načrtovanje

Kaj je načrtovanje IS?

- **Organizacija** programske opreme v module, komponente, razrede ali druge enote
 - Določanje **obnašanja** in **odgovornosti** posameznih modulov
 - Določanje **interakcije** in **vrste povezav** med posameznimi moduli oz. komponentami
- Tipične aktivnosti načrtovanja so:
 - Izdelava načrta arhitekture sistema
 - Izdelava načrta podatkovne baze
 - Izdelava načrta programskih modulov
 - Izdelava načrta dokumentacije
 - Izdelava načrta testiranja
 - Izdelava načrta namestitve in uvedbe

1. Načrtovanje arhitekture

- ▣ **Arhitekturni načrt** – načrtujemo, kako bo sistem porazdeljen po računalnikih ter kakšna strojna in programska oprema bo uporabljena za vsak računalnik
 - ▣ Določanje oz. izbira podsistemov, ki tvorijo celoten informacijski sistem
- ▣ Strojne in programske specifikacije
 - ▣ Podroben opis strojnih/programskih komponent je v pomoč odgovornim za nabavo teh izdelkov ter razvijalcem za razvoj IS

Arhitekturne komponente IS

- Podatkovna shramba (data storage)

- Dostop do podatkov (data access logic)

je logika, ki definira in omogoča dostop do shranjenih podatkov ter pravice dostopa

- Aplikacijska logika (application logic)

- Postopki obdelave podatkov IS (izvajanje funkcionalnosti)

- Predstavitev (presentation)

- Uporabniški vmesnik in prikaz informacij



Nekaj vrst programskih arhitektur (1/2)

□ Strežniška arhitektura (server-based)

- Uporabniki uporabljajo terminalski dostop, vse (razen samega prikaza) se izvaja **na strežniku**



□ Namizna/klientna arhitektura (client-based)

- Vsa aplikacija se izvaja **na odjemalcu**



Nekaj vrst programskih arhitektur (2/2)

- Arhitektura **odjemalec-strežnik** (client-server)
 - Izvajanje je porazdeljeno med **odjemalca** (običajno uporabniški vmesnik in preprostejša logika) in **strežnik** (zahtevnejša oz. poslovna logika, podatkovna baza)



- Te vrste je tudi **računalništvo v oblaku** (cloud computing)
 - Oblak predstavljajo strežniki podatkovne shrambe in aplikacijske logike
 - Strežniki so lahko **pri nas** (na naši strojni opremi) ali **pri ponudnikih oblačnih storitev** (npr. Amazon Web Services, Microsoft Azure, Google Cloud)
 - **IaaS** (Infrastructure as a Service), zakupite virtualni računalnik in z njim počnete kar želite
 - **PaaS** (Platform as a Service), zakupite možnost zagona specifične platforme (npr. le WordPress strani)

Arhitektura odjemalec-strežnik

▣ Strežnik (server)

- ▣ Skrbi za hrambo podatkov
- ▣ Največkrat skrbi tudi za logiko
 - Izvaja funkcionalnosti (poslovno logiko)
- ▣ Tipične tehnologije: Java, C#, Python, C, JavaScript(!)
- ▣ Razvijajo **back-end** razvijalci

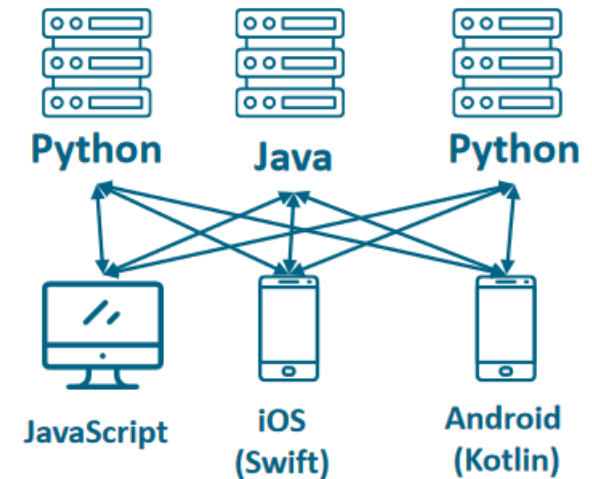
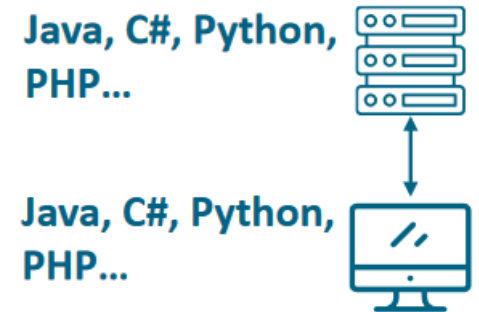
▣ Odjemalec (client)

- ▣ Skrbi za predstavitev
- ▣ Včasih skrbi za logiko
 - Največkrat so spletne strani le predstavitev
 - Vedno več je izvajanja logike
- ▣ Tipične tehnologije: HTML, PHP, Kotlin, Swift, JavaScript in njegove variante
- ▣ Razvijajo **front-end** razvijalci

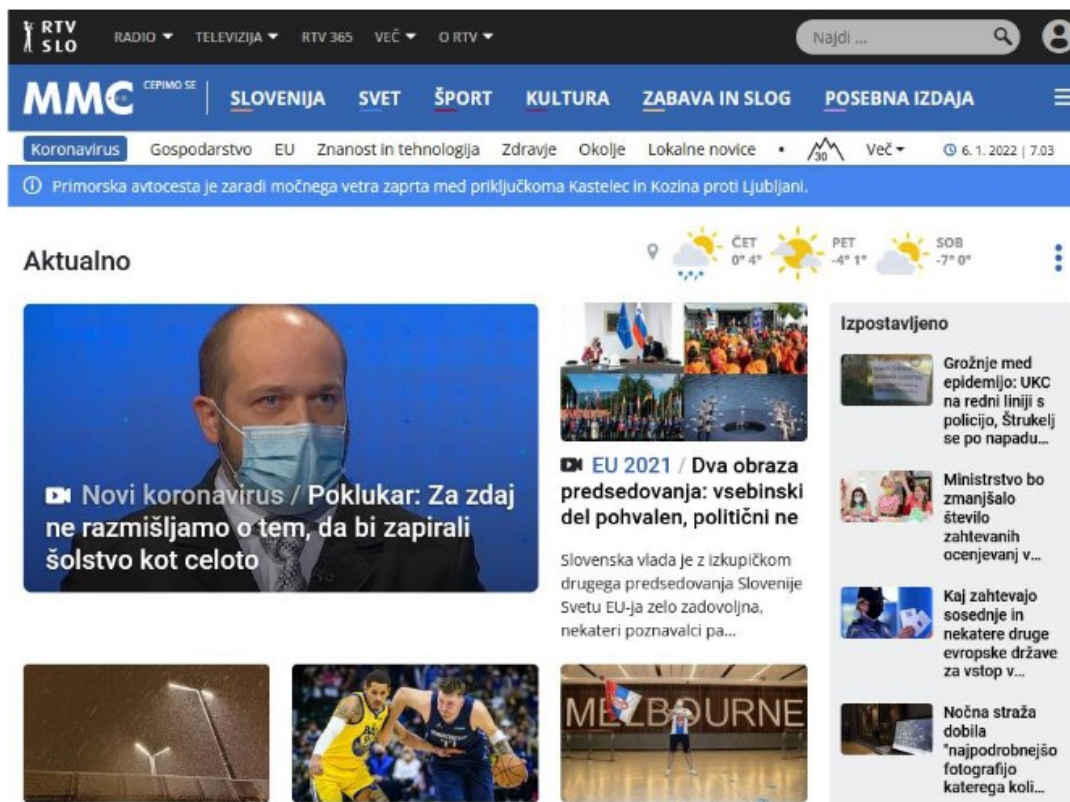
Obe strani (strežnik in odjemalca) obvladajo **full-stack** razvijalci

Arhitektura odjemalec-strežnik

- Če je tehnologija back-enda in front-enda enaka, govorimo o **monolitni** aplikaciji
 - Sklopljenost je močna (več o tem kasneje)
 - Majhna ekipa, ki obvlada le eno tehnologijo
 - Zahteve po le eni tehnologiji
 - Primerno za spletne aplikacije
- Če je tehnologija na obeh straneh različna (lahko tudi enaka), delovanje (in razvoj) vsake strani pa poteka ločeno, govorimo o **mikrostoritveni** aplikaciji
 - Šibka sklopljenost
 - Vsaka ekipa lahko uporabi svojo tehnologijo
 - Zahteva po podpori različnih platform (web, Android, iOS)
 - Vsak del back-enda ali front-enda je lahko v svoji tehnologiji



Dobro načrtovanje omogoča univerzalne rešitve



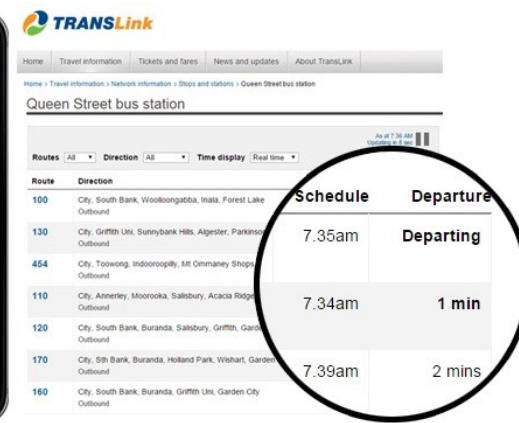
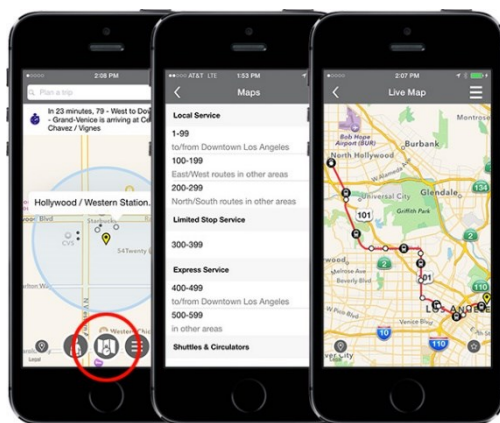
Prikaz v spletnem brskalniku na osebem računalniku.



Mobilna aplikacija.

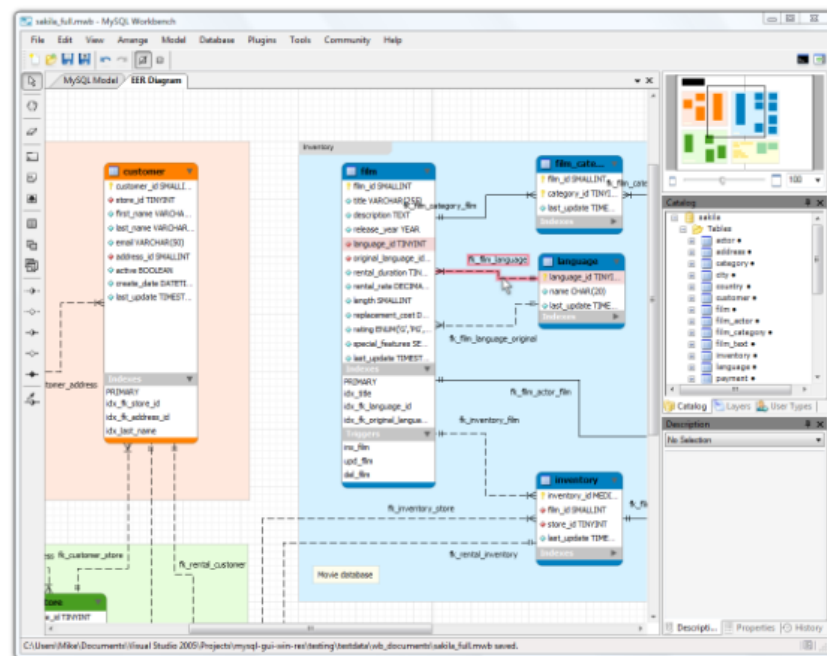
Vaja dela...

- Razviti želimo aplikacijo za spremljanje vozni redov mestnih avtobusov
 - Aplikacija naj omogoča uporabnikom pregledovanje vozni redov preko spleta in v obliki mobilne aplikacije
- Kako bi načrtovali arhitekturo tovrstne aplikacije?



2. Načrtovanje podatkovne baze

- Cilj je na podlagi konceptualnega modela (ER diagram) iz analize izdelati **logični podatkovni model** in izvesti ostale korake, potrebne za vzpostavitev učinkovite fizične podatkovne baze
- Rezultat:
 - Izdelamo **logični podatkovni model**
 - Določimo **sistem pravic** za uporabo podatkov

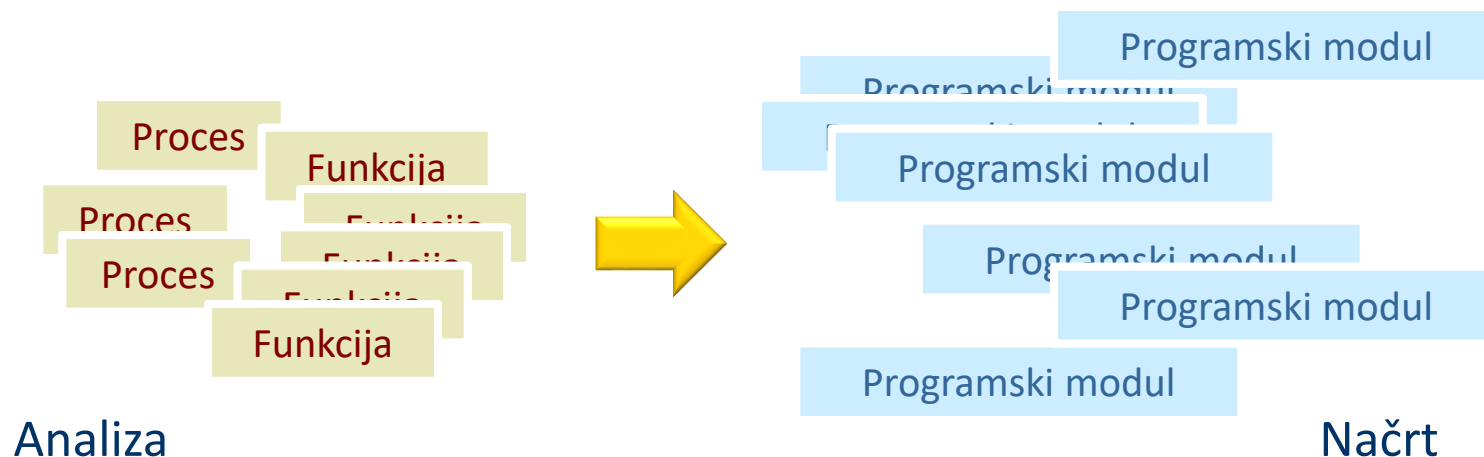


3. Načrtovanje programskih modulov

- Cilj je ustvariti navodila programerjem (koderjem), kako razdeliti celoten IS na posamezne programske module oz. komponente
- Namen je prikazati, kako bodo posamezne **funkcije in procesi**, identificirani v sklopu analize, **realizirani** v okviru IS
 - Top-down, modularni pristop: začnemo s celotnim sistemom, ki ga postopoma razgrajujemo in dodajamo podrobnosti
- Rezultat: načrt programskih modulov
 - Vse potrebne specifikacije, ki jih potrebujejo programerji, da implementirajo sistem

Načrtovanje programskih modulov

- Funkcije in procesi iz analize predstavljajo **logične sklope** sistema → v fazi načrtovanja jih pretvorimo v **programske sklope** ali **module** (bodoči fizični sklopi)
- Pretvorba ni nujno 1:1
 - Implementacija enega logičnega sklopa lahko zajema več modulov
 - En modul lahko implementira več logičnih enot



Zakaj programe delimo na module?

- Motivacija: vzdrževanje programske opreme predstavlja 60-80% celotne investicije v programsko opremo
- Razdelitev zagotavlja **lažje razumevanje, testiranje in popravljanje** informacijskega sistema
 - Lažje je locirati (in odstraniti) napake
 - Popravki so običajno lokalne narave, s čimer dosežemo večjo stabilnost IS
 - S popravki v enem modulu ne pokvarimo delovanja preostalega dela programa
 - Lažja je prenosljivost posameznih komponent v druge programe
 - Ponovna uporaba komponent je višja
- Razvoj IS je lahko skupinski – vsaka skupina razvija svojo komponento

Kompleksnost problema

□ Izzivi:

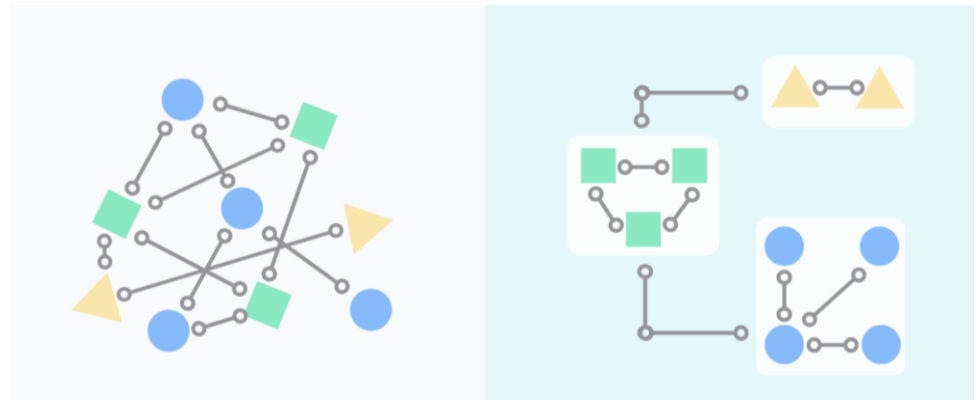
- Kompleksnost sestavljenega problema je večja kot vsota kompleksnosti posameznih problemov
- Če problem razdelimo na preveliko število podproblemov, prekomerno naraste trud za vzpostavitev potrebnih povezav
- Najti je potrebno **optimalno delitev** programskih modulov oz. komponent

□ Cilji:

- **Zagotoviti** relativno **neodvisnost** komponent
- **Omogočiti** hitro in učinkovito **razumevanje odvisnosti**
- **Identificirati** nejasne in skrite **povezave** med komponentami

Optimalna delitev

- Težimo k **funkcionalni neodvisnosti** modulov oz. komponent
- Dve merili za kvalitativno vrednotenje načrtovanja
 - **Sklopljenost** (coupling) – merilo relativne odvisnosti modulov oz. komponent
 - **Vezljivost** (cohesion) – merilo funkcionalne moči modula oz. komponente



slaba delitev

dobra delitev

Sklopljenost

Šibka sklopljenost
(najboljše)



Močna sklopljenost
(najslabše)

- **Merilo odvisnosti** modulov ali komponent med seboj – opazujemo **povezavo enega modula do drugih**
- Komponenti ali modula sta popolnoma neodvisna, če vsak deluje v celoti pravilno brez prisotnosti drugega
- Sklopljenost predstavlja **verjetnost**, da moramo v fazi implementacije pri popravljanju ali **spreminjanju enega** modula ali komponente **upoštevati** neke značilnosti ali **lastnosti drugega** modula

1. **Neodvisna** – ni nobene povezanosti med moduli
2. **Podatkovna** – komunikacija med moduloma (klicanje) z uporabo parametrov
3. **Pečatna** – moduli si delijo del skupne podatkovne strukture
4. **Nadzorna** – en modul nadzira delovanje drugega (z uporabo kontrolnih zastavic)
5. **Zunanja** – za delovanje se moduli zanašajo na skupni eksterni sistem oz. I/O enoto
6. **Skupna** – skupni globalni podatki
7. **Vsebinska** – en modul dostopa neposredno do vsebine drugega, npr. souporaba kode

Vezljivost

Visoka vezljivost
(najboljše)



Nizka vezljivost
(najslabše)

- **Merilo povezanosti** modula ali komponente – opazujemo **notranjo sestavo** modula
- Predstavlja smiselnost funkcionalnosti, ki jih en modul oz. komponenta naslavlja
 - Ima smisel imeti modul?
 - Imajo elementi modula kaj skupnega?
- Želimo čim bolj specializirane module oz. komponente, saj lahko take najlažje ponovno uporabimo

1. **Funkcijska** – celoten modul je namenjen izvedbi točno določenega, zaključenega opravila
2. **Sekvenčna** – izhod enega elementa v modulu je vhod v drugega
3. **Komunikacijska** – elementi v modulu uporabljajo iste podatke
4. **Proceduralna** – elementi so uvrščeni v modul glede na zaporedje izvajanja funkcij
5. **Časovna** – elementi, ki se morajo izvesti v nekem času
6. **Logična** – elementi s sorodnimi funkcijami, ponavadi izberemo želeno s pošiljanjem zastavice
7. **Naključna** – elementi v modulu niso vsebinsko sorodni med seboj (npr. nabor splošnih funkcij)

Vaja dela...

- Kako bi načrtovali programske “module” za naslednjo nalogo:
 - Izdelajte programsko rešitev, ki od uporabnika prebere naravno število n , izračuna fakulteto tega števila $n!$ in rezultat izpiše
 - Zaradi enostavnosti primera, si predstavljajte “module” kot funkcije v izbranem programskem jeziku (ali v psevdo-kodi)
 - Nalogo dopolnite tako, da lahko uporabnik vnese še drugo število k , program pa izračuna ter izpiše vrednost binomskega števila $\begin{bmatrix} n \\ k \end{bmatrix}$

Katere lastnosti imajo “dobri” moduli?

- Vsak modul ima **natančno definirano nalogo**
 - Obstaja neposredna povezava med funkcionalnostmi sistema in funkcijami modula (visoka, funkcijska vezljivost)
- Modul lahko **uporabimo** (testiramo) v izolaciji **brez drugih** modulov
 - Na delovanje posameznega modula drugi moduli ne vplivajo, ali le zelo malo (šibka, neodvisna sklopljenost)
- Ni potrebe, da se natančno pregleda notranjost modula, da bi razumeli funkcijo, ki jo opravlja
- Moduli se lahko kombinirajo v skupne večje module

Načrtovalski vzorci

- Ker ne obstaja metoda, ki bi neposredno (korak za korakom) pripeljala od analize do načrta, so za dobro načrtovanje ključnega pomena izkušnje
- Da bi omogočili “uporabo znanja” izkušenih razvijalcev tudi drugim, uporabljamo **načrtovalske vzorce** (design patterns)
- Načrtovalski vzorci predstavljajo **jedro rešitve** za običajne, splošne izzive, ki nastopajo pri načrtovanju
 - Ko pri razvoju naletimo na primer, ki ga želimo rešiti, uporabimo načrtovalski vzorec, ki narekuje pristop k rešitvi

4. Načrt dokumentacije

- Namen aktivnosti je določiti **obseg in strukturo dokumentacije** ter izbrati ustrezne standarde in vzorce za dokumentacijo
 - Upoštevamo zahteve naročnika iz zajema in specifikacije zahtev
- V grobem lahko dokumentacijo razdelimo na:
 - **Uporabniško** dokumentacijo (kako uporabljati IS)
 - **Sistemske-tehnično** dokumentacijo (kako namestiti IS)
 - Navodila za **operativno skrbništvo** (kako upravljati IS)

5. Načrt testiranja

- Testiranje, ki intenzivno poteka v fazi implementacije ter v okviru namestitve in uvedbe sistema, je nujno predhodno načrtovati
 - Načrt testiranja **je odvisen** od načrta programskih modulov
- **Testni primeri**
 - Testni primer je **en test, ki ga bomo izvedli**
 - Definirati je potrebno namen tega testa, pričakovane in dobljene rezultate ter uspešnost/neuspešnost testa
- **Scenarij testiranja**
 - Podrobno določimo **zaporedje izvajanja testnih primerov** po sklopih IS in po okoljih
 - Vsebuje vse potrebne informacije o potrebnih in izvedenih testnih primerih ter njihovih rezultatih

6. Načrt namestitve in uvedbe

- Naloga aktivnosti je izdelati **načrt namestitve in uvedbe IS v razvojno, testno in produkcijsko okolje** ter uvedbo uporabnikov in skrbnikov za delo z IS
- **Razvojno okolje** (development): Računalniki na katerih se programira in računalniki, ki pomagajo razvijalcem pri razvoju
 - Načrt namestitve in uvedbe v razvojnem okolju pripravijo člani projektne skupine razvijalca
- **Testno okolje** (testing): Računalniki na katerih poteka testiranje
- **Produkcijsko okolje** (production): Računalniki na katerih se izvaja IS, ko se uporablja v realnem svetu
 - Pri izdelavi načrta namestitve in uvedbe v testnem oz. produkcijskem okolju sodelujejo tudi predstavniki končnih uporabnikov

Predstavitev načrta

- Osnovni namen predstavitve je pridobitev potrditve s strani naročnika o ustreznosti načrta
 - Pri predstavitvi je potrebno upoštevati, da naročnik navadno nima strokovnih znanj, ki bi mu omogočala poglobljeno razumevanje načrta, hkrati pa to tudi ni njegova naloga
 - Naročniku predstavimo le osnovne elemente načrta
 - Primeren za podrobno predstavitev je načrt uporabniškega vmesnika → posredno razkriva tudi druge elemente načrta
 - Na osnovi predstavitve lahko naročnik opozori na pomanjkljivosti oz. izrazi dodatne želje